JOMHACK
EMPOWERING INNOVATION

```
> bootcamp.learn()
```

JomCode Coding Bootcamp

CHAPTER 14

FastAPI, Uvicorn and Swagger fundamentals

`learn / type / run / explain`

Plain meaning and technical explanation

PLAIN MEANING

An API function handles a request and returns a response.

TECHNICAL EXPLANATION

FastAPI maps decorators to HTTP operations. Pydantic validates typed input. Uvicorn serves the ASGI application; Swagger UI calls its endpoints.

REAL-WORLD USE

Create a small post API and verify its contract before adding a database.

QUICK TIP

Keep the terminal running while using http://localhost:8000/docs. Ctrl+C stops the server.

INSTALL / RUN

Install: python -m pip install fastapi uvicorn. Windows: py -m uvicorn workbook:app --reload. macOS: python3 -m uvicorn workbook:app --reload. Run from this chapter folder.

Python environment and first API process

Create workbook.py in this chapter folder. The following shared setup creates app before decorators use it.

Windows: py -m venv .venv; use .venv\Scripts\python.exe for pip and Uvicorn.

macOS: python3 -m venv .venv; use .venv/bin/python for pip and Uvicorn.

Install the packages, keep terminal A running, then open http://localhost:8000/docs.

```
python -m pip install fastapi uvicorn
python -m uvicorn workbook:app --reload --port 8000
```

Shared setup

workbook.py

```
01    from fastapi import FastAPI, HTTPException
02    from pydantic import BaseModel, Field
03    app = FastAPI(title="JomCode Chapter 14")
```

PRO TIP Keep this setup once at the top of the file.

Worked example 1: Health endpoint

workbook.py

```
01    @app.get("/health")
02    def health():
03        return {"ok": True}
```

PRO TIP Start Uvicorn from the folder containing workbook.py.

Run and explain example 1

EXPECTED RESULT

Swagger: GET /health returns 200 and {"ok": true}.

WHY IT BEHAVES THIS WAY

The decorator registers this function for the method and path. FastAPI serializes the dictionary to JSON.

Worked example 2: Path and query parameters

workbook.py

```
@app.get("/posts/{post_id}")
def read_post(post_id: int, preview: bool = False):
    return {"id": post_id, "preview": preview}
```

PRO TIP In Swagger, expand the operation, select Try it out, enter values, then Execute.

Run and explain example 2

EXPECTED RESULT

GET /posts/7?preview=true returns 200 with id=7 and preview=true. /posts/abc returns 422.

WHY IT BEHAVES THIS WAY

Annotations convert and validate HTTP text before the function runs.

Worked example 3: Validated request body

workbook.py

```
class PostInput(BaseModel):
    title: str = Field(min_length=1, max_length=160)
    body: str = Field(min_length=1, max_length=20000)

@app.post("/posts", status_code=201)
def create_post(payload: PostInput):
    return {"id": 1, **payload.model_dump()}
```

PRO TIP Length validation alone allows whitespace. Add strip() rules in the service layer.

Run and explain example 3

EXPECTED RESULT

POST /posts with {"title":"Hi","body":"Welcome"} returns 201. An empty title returns 422.

WHY IT BEHAVES THIS WAY

Pydantic checks the body before the handler receives it. This example returns data without persisting it.

Worked example 4: Explicit HTTP error

workbook.py

```
@app.get("/missing")
def missing():
    raise HTTPException(404, "Post not found")
```

PRO TIP A traceback and status 500 usually indicate an unexpected server failure.

Run and explain example 4

EXPECTED RESULT

GET /missing returns 404 and {"detail":"Post not found"}.

WHY IT BEHAVES THIS WAY

HTTPException turns an expected application failure into a documented HTTP response.

Exercise 1: Greeting endpoint

YOUR TASK

Add GET /greeting with a name query parameter defaulting to Reader.

CHECK THESE CASES

No name gives Hello, Reader. name=Ada gives Hello, Ada.

HINT AFTER A TRY

Use a typed function argument with a default.

Exercise 2: Whitespace validation

YOUR TASK

Add POST /checked-posts and reject a title containing only whitespace.

CHECK THESE CASES

Blank title returns 422. " Hi " returns cleaned title Hi.

HINT AFTER A TRY

Strip the title and raise HTTPException when empty.

Checkpoint before moving on

01 Run and explain the four worked examples, including their shared setup.

02 Attempt both exercises before reading their solutions. Check the stated acceptance cases.

03 Explain the real-world connection: Create a small post API and verify its contract before adding a database.

Official reference